

Neural Networks I: Introduction to Neural Networks

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



From Lecture 16

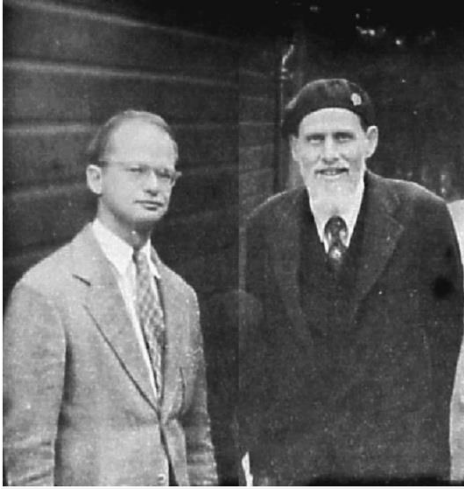
Imbalanced Classification

- *Why accuracy alone is not enough for imbalanced data*
- *ROC curves, ROC-AUC, and precision-recall curves*
- *Class weighting for logistic regression, SVM, and random forests*

In this lecture, you will learn

- *What a neural network (NN) is and how it is structured*
- *What activation functions do and why nonlinearity matters*
- *What TensorFlow and Keras are and how they are used for NNs*

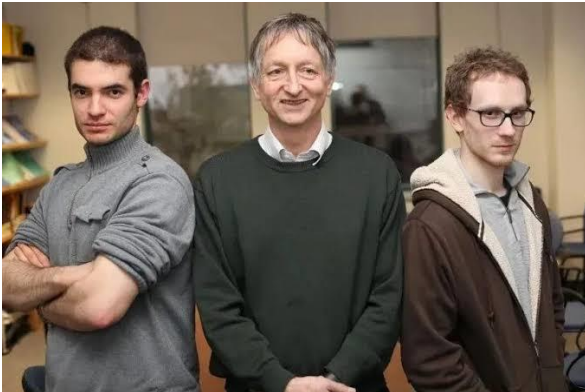
Brief History of Neural Networks



In 1943, at the University of Chicago, Warren McCulloch and Walter Pitts introduced the first mathematical model of a neural network



Frank Rosenblatt with a Mark I perceptron computer in 1960.

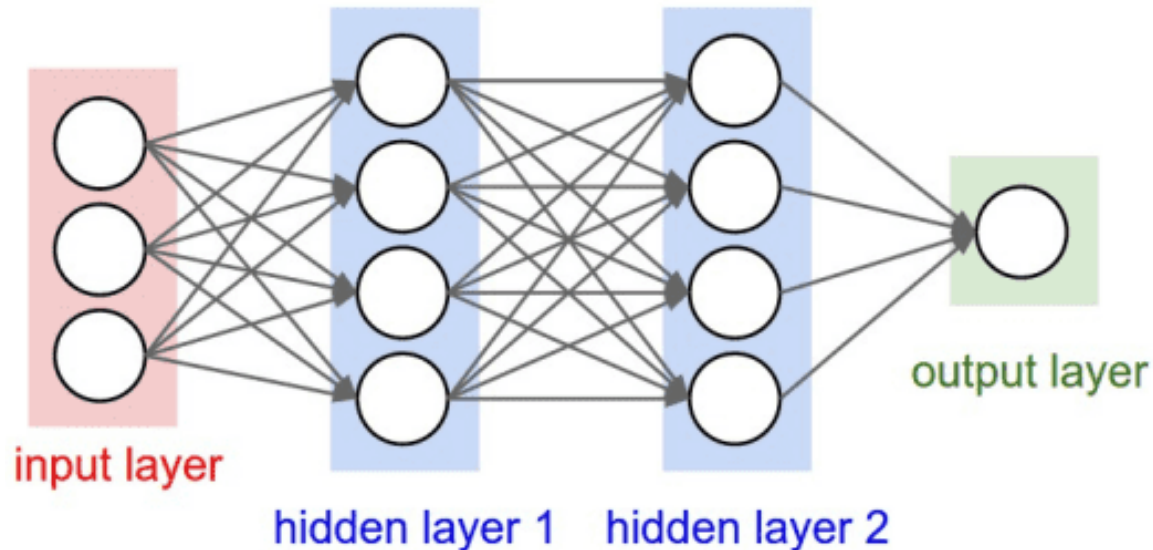


University of Toronto professor Geoffrey Hinton (center, “The Godfather of AI”), and graduate student Ilya Sutskever (left, co-founded OpenAI), and Alex Krizhevsky (right, “unsung hero of AI”).

- **1943:** McCulloch & Pitts propose the first artificial neuron model.
- **1958:** Rosenblatt introduces the **Perceptron**, an early learning algorithm.
- **1986:** **Backpropagation** is popularized by Rumelhart, Hinton, and Williams to “train” a neural network.
- **1990s:** Neural networks are applied to **speech recognition** and **handwritten digit recognition**.
- **2012:** **AlexNet** wins the ImageNet competition using CNNs → major breakthrough in computer vision.
- **2014–2017:** Major advances include **GANs**, **LSTMs**, **attention**, and **Transformers**.
- **2018–present:** Large language models such as **BERT** and **GPT** become popular to AI research and applications.

Overview of Neural Network

- **Neural networks (NNs)** are computational models inspired by the brain, built from layers of interconnected neurons.
- They learn by adjusting **weights** and **biases** during training, allowing them to discover complex, **nonlinear** patterns in data.



Multilayer Perceptron (Feed-forward neural network, nonlinear regression)

Architecture

- **Input layer:** takes in the data
- **Hidden layers:** learn useful patterns and features
- **Output layer:** gives the final prediction

Building Blocks

- **Neurons (nodes):** basic computing units
- **Weights & biases:** control how strongly information is passed
- **Activation functions:** add nonlinearity

Training

- **Loss function:** measures how wrong the prediction is
- **Backpropagation:** adjusts weights to reduce the error

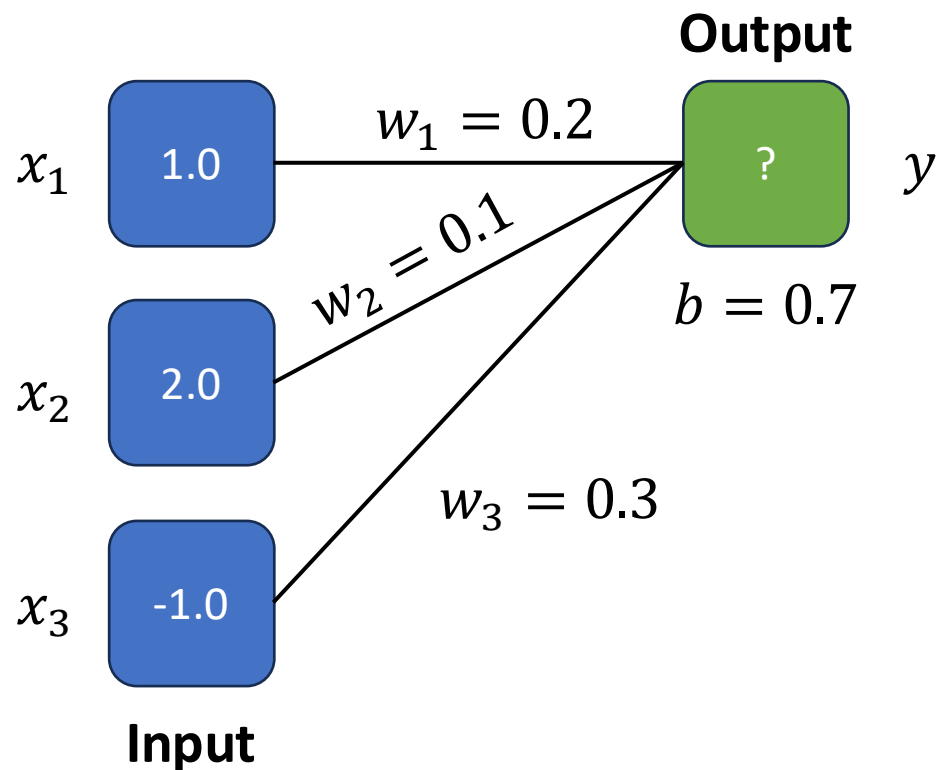
Today's lecture

Nodes and Hidden Layers

Let's start with a familiar model: linear regression.

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

We can draw this equation as a simple network: 1) x_1, x_2, x_3 are input neurons (blue), 2) w_1, w_2, w_3 are weights, 3) b is bias, 4) y is output neuron (green).



What is y ?

$$\begin{aligned} y &= w_1x_1 + w_2x_2 + w_3x_3 + b \\ &= 0.2 \times 1.0 + 0.1 \times 2.0 + 0.3 \\ &= 0.2 + 0.2 - 0.3 \\ &= 0.1 \end{aligned}$$

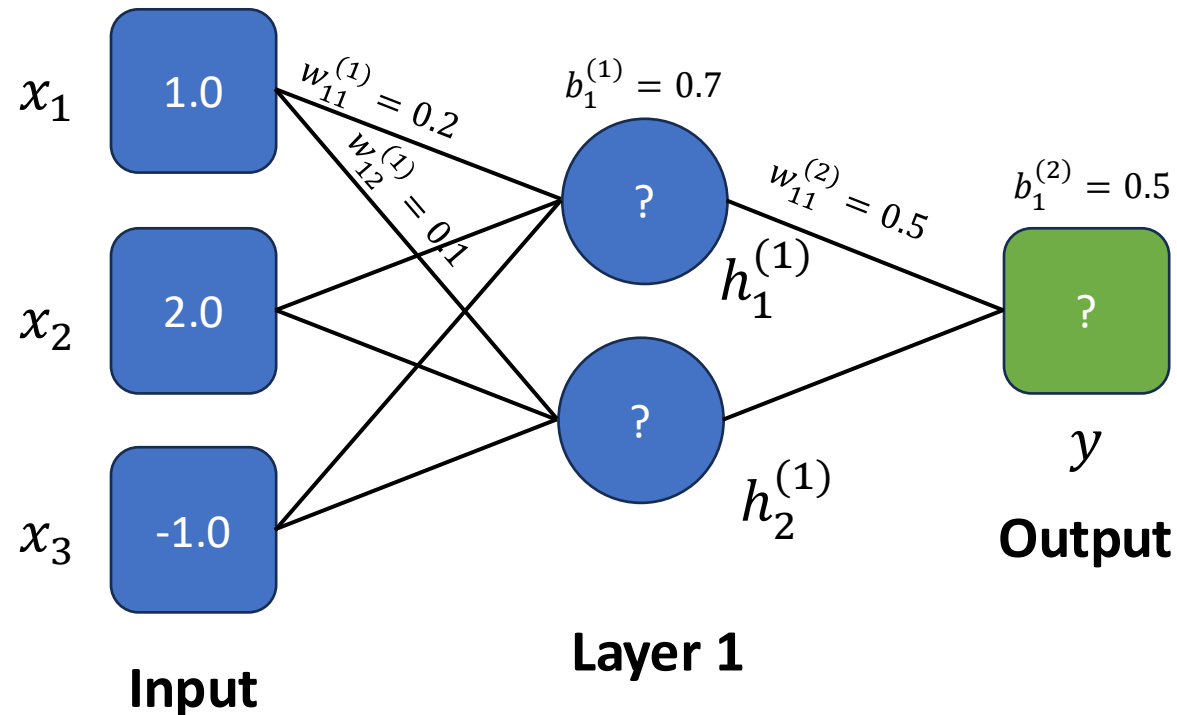
Nodes and Hidden Layers

Now let's add one hidden layer with two neurons.

Each hidden neuron receives all three inputs, but uses its own set of weights of and bias.

Definitions:

- $w_{ij}^{(l)}$: l layer being entered, i source neuron, j destination neuron.
- $b_j^{(l)}$: l layer being entered, j destination neuron
- $h_j^{(l)}$: l hidden layer number, j neuron number in that layer

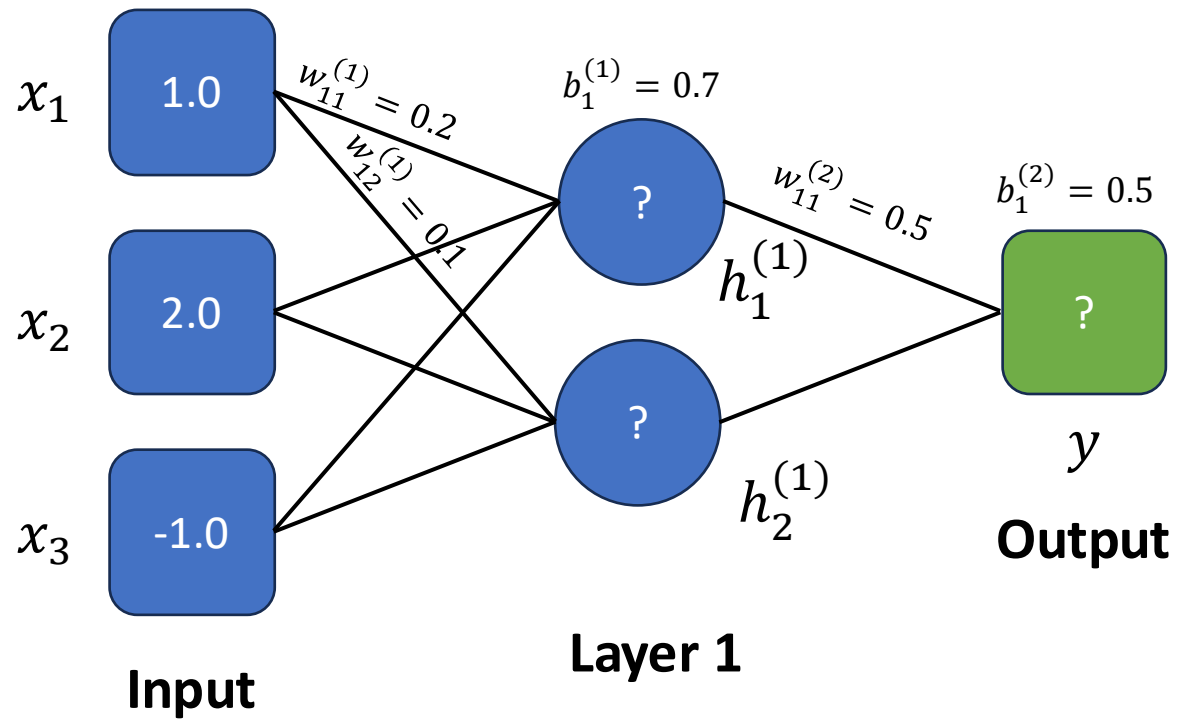


What does $w_{22}^{(1)}$ mean?

What does $b_2^{(1)}$ mean?

What does $h_2^{(1)}$ mean ?

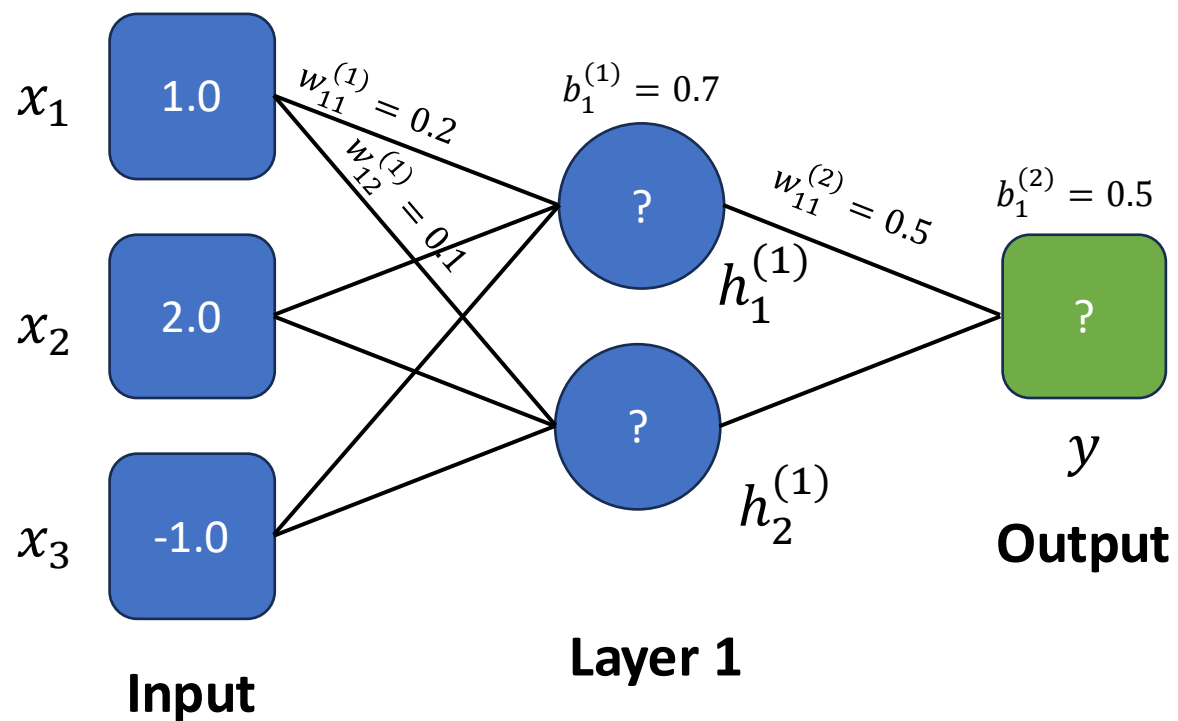
Nodes and Hidden Layers



Weight / bias	Value
$w_{11}^{(1)}$	0.2
$w_{12}^{(1)}$	0.1
$w_{21}^{(1)}$	0.3
$w_{22}^{(1)}$	0.4
$w_{31}^{(1)}$	-0.5
$w_{32}^{(1)}$	0.3
$b_1^{(1)}$	0.7
$b_2^{(1)}$	-0.9

What are the values of the two hidden neurons, $h_1^{(1)}$ and $h_2^{(1)}$?

Nodes and Hidden Layers

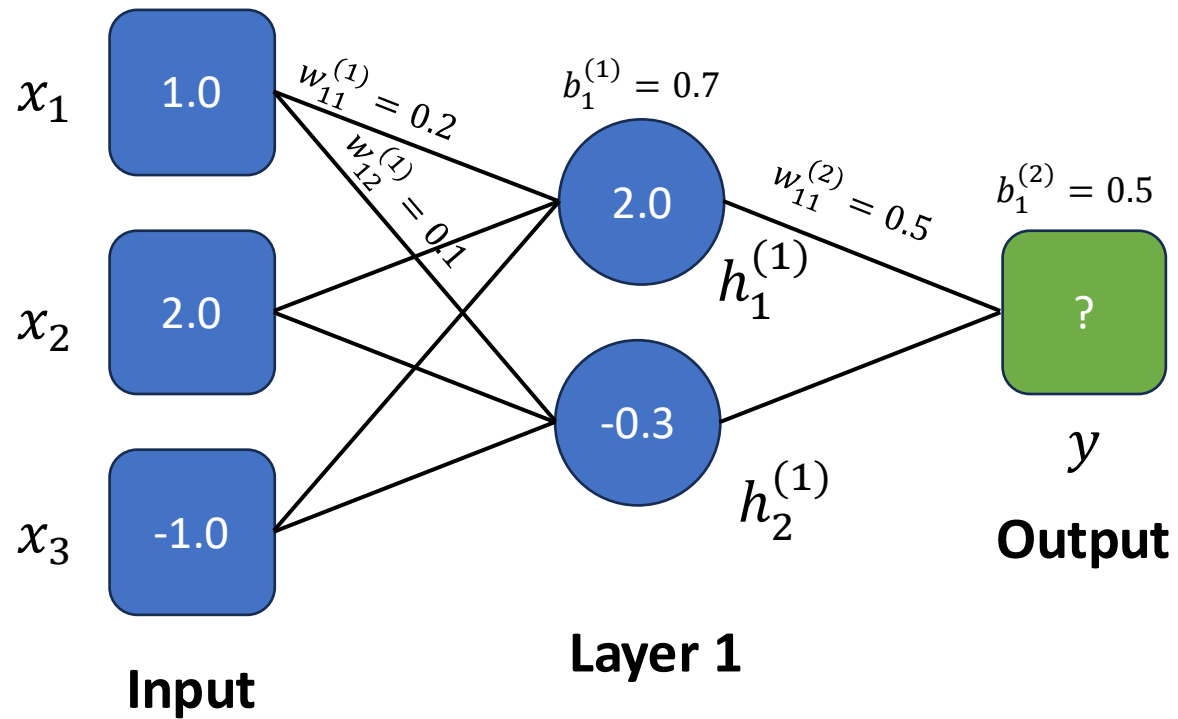


Weight / bias	Value
$w_{11}^{(1)}$	0.2
$w_{12}^{(1)}$	0.1
$w_{21}^{(1)}$	0.3
$w_{22}^{(1)}$	0.4
$w_{31}^{(1)}$	-0.5
$w_{32}^{(1)}$	0.3
$b_1^{(1)}$	0.7
$b_2^{(1)}$	-0.9

$$\begin{aligned}h_1^{(1)} &= w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + w_{31}^{(1)}x_3 + b_1^{(1)} \\&= (0.2)(1.0) + (0.3)(2.0) + (-0.5)(-1.0) + 0.7 \\&= 0.2 + 0.6 + 0.5 + 0.7 = \boxed{2.0}\end{aligned}$$

$$\begin{aligned}h_2^{(1)} &= w_{12}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{32}^{(1)}x_3 + b_2^{(1)} \\&= (0.1)(1.0) + (0.4)(2.0) + (0.3)(-1.0) - 0.9 \\&= 0.1 + 0.8 - 0.3 - 0.9 = \boxed{-0.3}\end{aligned}$$

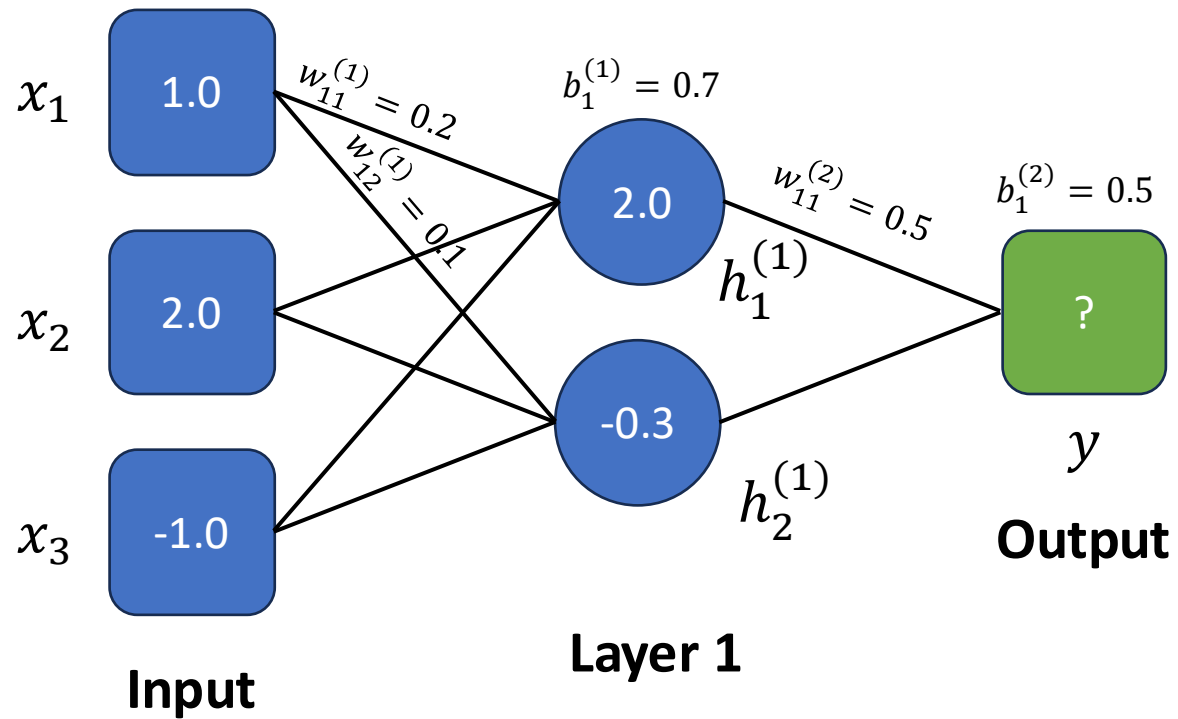
Nodes and Hidden Layers



Weight / bias	Value
$w_{11}^{(2)}$	0.5
$w_{21}^{(2)}$	0.4
$b_1^{(2)}$	-0.3

What is the value of the output neurons, y ?

Nodes and Hidden Layers



What is the value of the output neurons, y ?

Weight / bias	Value
$w_{11}^{(2)}$	0.5
$w_{21}^{(2)}$	0.4
$b_1^{(2)}$	-0.3

$$\begin{aligned} y &= w_{11}^{(2)} h_1^{(1)} + w_{21}^{(2)} h_2^{(1)} + b_1^{(2)} \\ &= (0.5)(2.0) + (0.4)(-0.3) - 0.3 \\ &= 1.0 - 0.12 - 0.3 \\ &= \boxed{0.58} \end{aligned}$$

Activation Functions

- Adding hidden layers alone does **not** create nonlinearity.
- A sequence of linear operations is still just a **linear transformation**.

$$\begin{aligned}h_1^{(1)} &= w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + w_{31}^{(1)}x_3 + b_1^{(1)} \\&= (0.2)(1.0) + (0.3)(2.0) + (-0.5)(-1.0) + 0.7 \\&= 0.2 + 0.6 + 0.5 + 0.7 = \boxed{2.0}\end{aligned}$$

$$\begin{aligned}h_2^{(1)} &= w_{12}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{32}^{(1)}x_3 + b_2^{(1)} \\&= (0.1)(1.0) + (0.4)(2.0) + (0.3)(-1.0) - 0.9 \\&= 0.1 + 0.8 - 0.3 - 0.9 = \boxed{-0.3}\end{aligned}$$

Matrix Form

$$\mathbf{h}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\begin{bmatrix} h_1^{(1)} \\ h_2^{(1)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \end{bmatrix}$$

Activation Functions

- Adding hidden layers alone does **not** create nonlinearity.
- A sequence of linear operations is still just a **linear transformation**.

$$\begin{aligned}y &= w_{11}^{(2)} h_1^{(1)} + w_{21}^{(2)} h_2^{(1)} + b_1^{(2)} \\&= (0.5)(2.0) + (0.4)(-0.3) - 0.3 \\&= 1.0 - 0.12 - 0.3 \\&= \boxed{0.58}\end{aligned}$$

Matrix Form

$$\begin{aligned}y &= \mathbf{W}^{(2)} \mathbf{h}^{(1)} + b_1^{(2)} \\&= \begin{bmatrix} w_{11}^{(2)} & w_{21}^{(2)} \end{bmatrix} \begin{bmatrix} h_1^{(1)} \\ h_2^{(1)} \end{bmatrix} + b_1^{(2)}\end{aligned}$$

Final Matrix Form

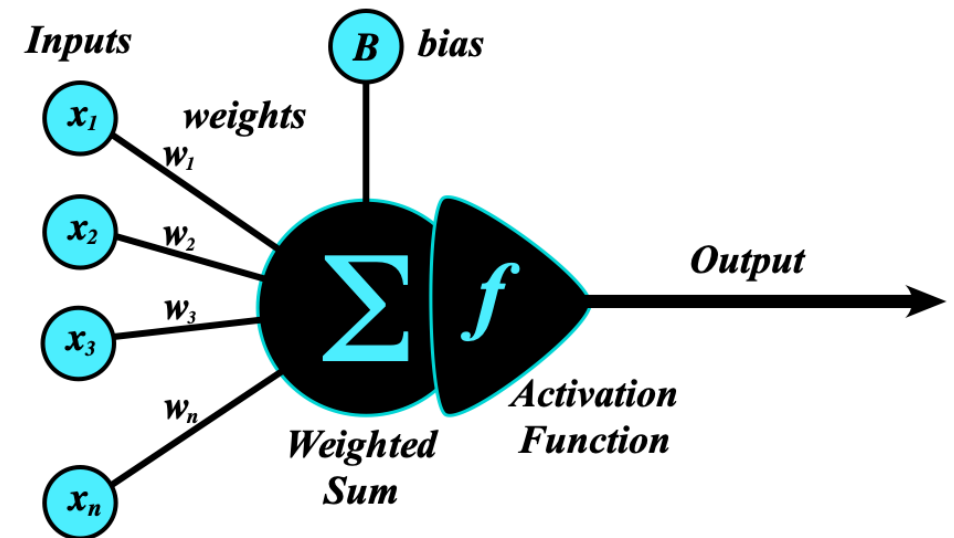
$$\begin{aligned}y &= \mathbf{W}^{(2)} \mathbf{h}^{(1)} + \mathbf{b}^{(2)} \\&= \mathbf{W}^{(2)} \left(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)} \\&= \left(\mathbf{W}^{(2)} \mathbf{W}^{(1)} \right) \mathbf{x} + \left(\mathbf{W}^{(2)} \mathbf{b}^{(1)} + \mathbf{b}^{(2)} \right)\end{aligned}$$

Is this linear or nonlinear?

Activation Functions

- To model nonlinear relationships, we apply an **activation function** to each neuron's output.
- This nonlinear output is then passed to the next layer.

Figure sources: makeagif.com, mriquestions.com/what-is-a-neural-network.html



Like a biological neuron that fires an action potential when sufficiently stimulated, an activation function determines an artificial neuron's output.

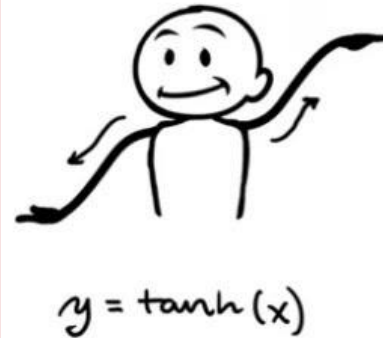
Common Activation Functions

Activation functions transform a neuron's weighted sum, z , into its output, $a = f(z)$.

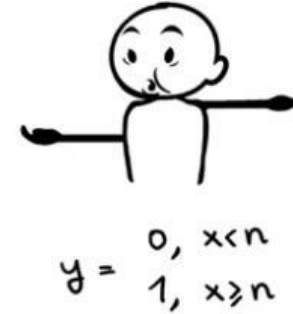
Sigmoid



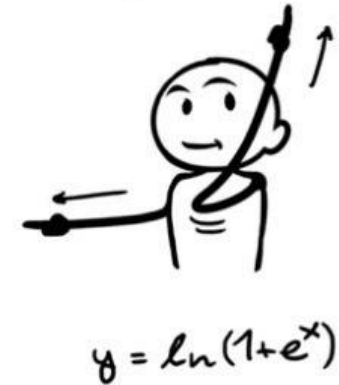
Tanh



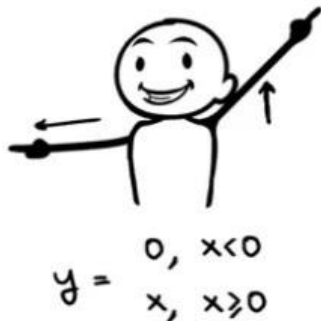
Step Function



Softplus



ReLU



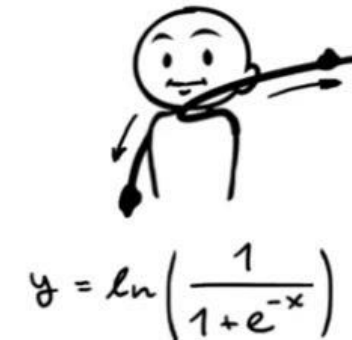
Softsign



ELU

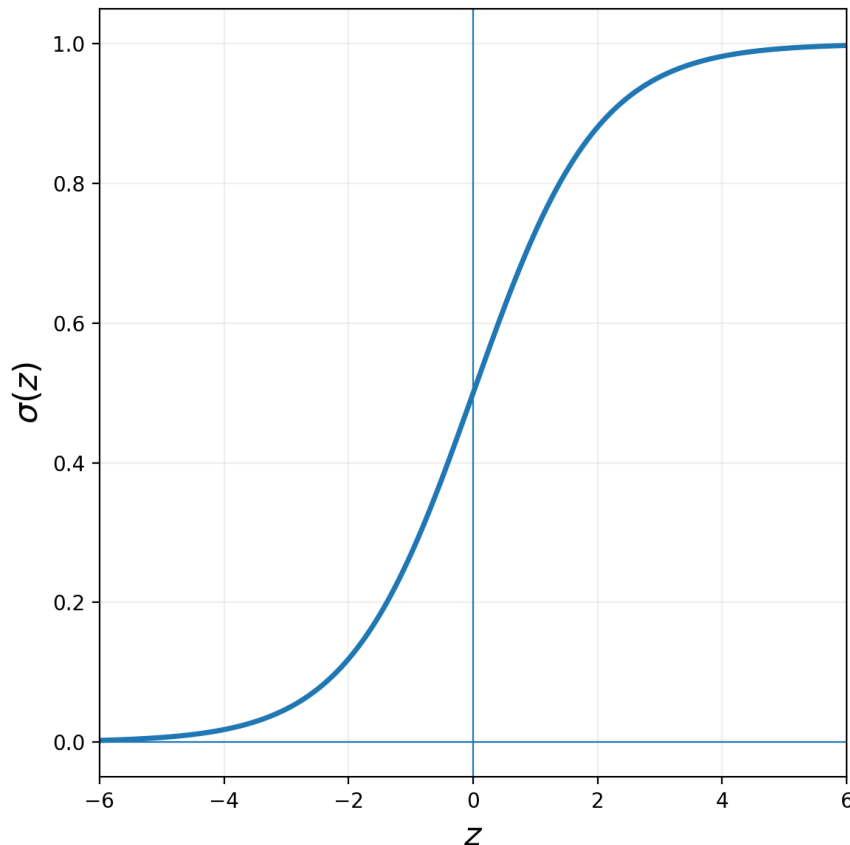


Log of Sigmoid



Sigmoid

- Maps any input z into **0 to 1**
- Larger $z \rightarrow$ output closer to **1**
- Smaller $z \rightarrow$ output closer to **0**
- Commonly used in the output layer for **binary classification**



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Example: if $z = 2$,

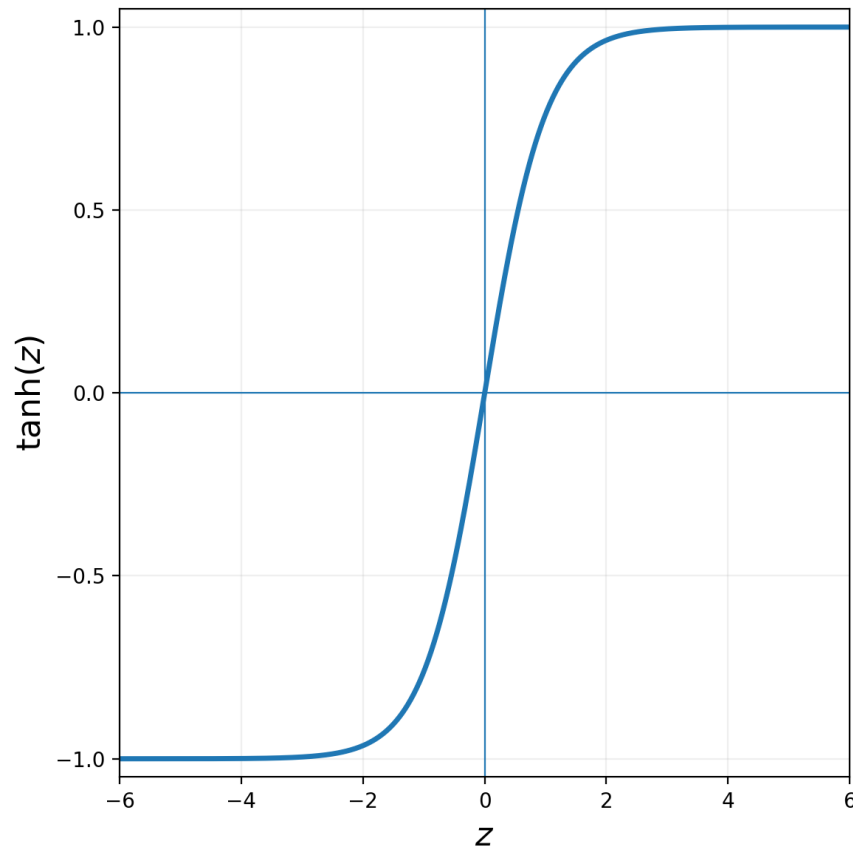
$$\sigma(2) = 0.88$$

which can be interpreted as an 88% predicted probability for class 1.

Which has a higher probability of being class 1, $z = -3$ or $z = 4$?

tanh

- Maps any input z into **-1 to 1**
- Centered at **0**
- Preserves the sign: negative $z \rightarrow$ negative output, positive $z \rightarrow$ positive output
- Often used when we want **zero-centered hidden activations**



$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Example:

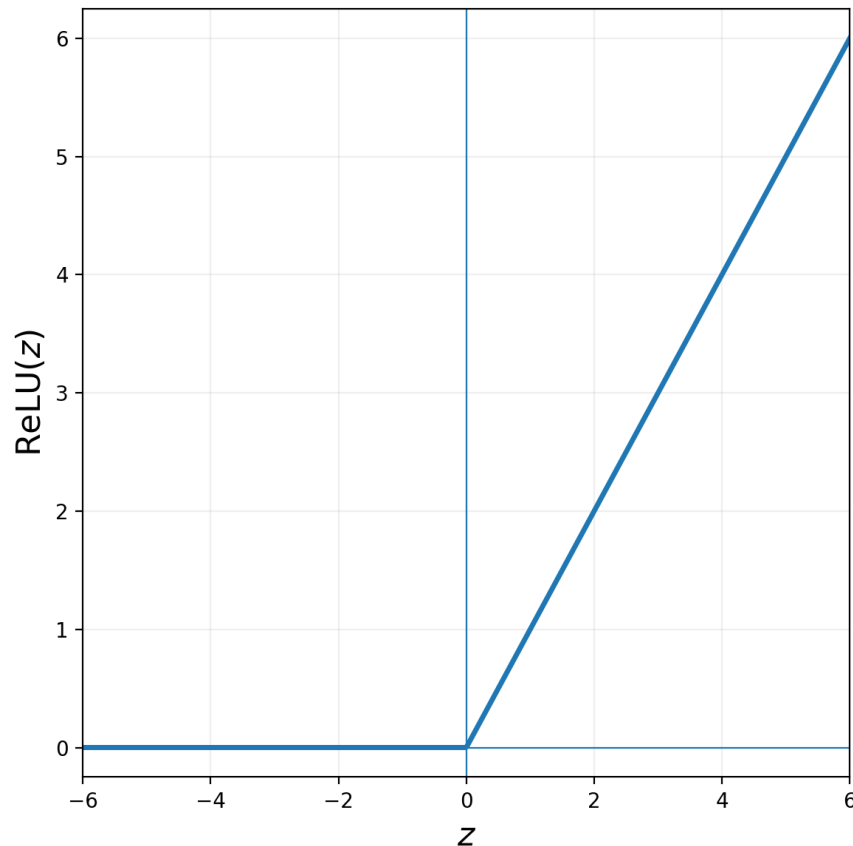
If $z = 2$, $\tanh(2) = 0.96$

What is $\tanh(-2)$?

How about $\tanh(0)$?

ReLU

- Negative $z \rightarrow 0$
- Positive $z \rightarrow$ **unchanged**
- No upper bound: large positive inputs stay large
- **Most commonly used in hidden layers**



$$\text{ReLU}(z) = \max(0, z)$$

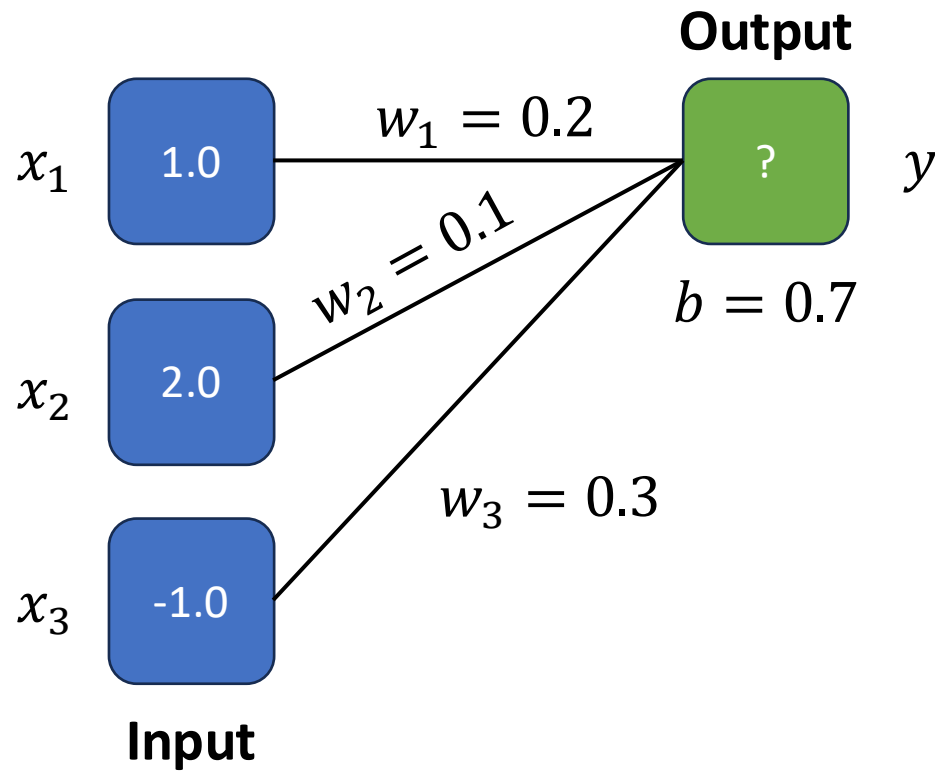
Example:

If $z = 2$, $\text{ReLU}(2) = 2$

What is $\text{ReLU}(-2)$?

Why do we call it a nonlinear activation function? It has only linear components.
ReLU is piecewise linear, but it is not one single linear function.

Activation Function Exercise



Apply a sigmoid activation function to the output

$$\begin{aligned} y &= \sigma(w_1x_1 + w_2x_2 + w_3x_3 + b) \\ &= \sigma(0.1) \\ &\approx 0.52 \end{aligned}$$

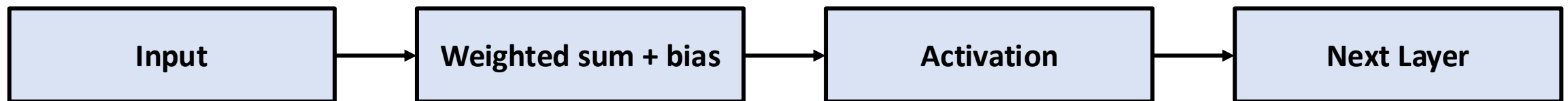
Apply a ReLU activation function to the output

$$\begin{aligned} y &= \text{ReLU}(w_1x_1 + w_2x_2 + w_3x_3 + b) \\ &= \text{ReLU}(0.1) \\ &= 0.1 \end{aligned}$$

A Quick Summary

A neural network is built from three key ingredients:

- **Layers of neurons:** input layer, one or more hidden layers, and an output layer.
- **Weights and biases:** trainable parameters that determine how information is combined between layers.
- **Activation functions:** nonlinear transformations applied to neuron outputs before passing them to the next layer.



This process is called “**forward propagation**,” which is the process of passing inputs through the network, layer by layer, to produce an output.

What does a weight w do in a neural network?

- A. Determines how strongly an input influences a neuron
- B. Measures the prediction error
- C. Adds nonlinearity
- D. Determines the number of layers

Answer: A. Bias shifts the neuron's weighted sum, while weights control the contribution of each input.

A neural network has several hidden layers, but no activation functions. What happens?

- A. More layers automatically make the model nonlinear
- B. The network is still equivalent to one linear transformation
- C. Bias terms create the needed nonlinearity
- D. It depends only on the number of neurons

Answer: B. Without nonlinear activation functions, multiple layers can collapse into one linear/affine transformation.



TensorFlow

A full machine-learning framework for building, training, and deploying neural networks.

**Install by
next class**



Keras

A high-level, user-friendly API for quickly creating and training neural networks.

No too much coding detail

```
conda activate eche589
```

```
python -m pip install tensorflow keras
```



PyTorch

A flexible deep-learning framework widely used in computer science research and increasingly in production.

In this lecture, we learned

- *What a neural network (NN) is and how it is structured*
- *What activation functions do and why nonlinearity matters*
- *What TensorFlow and Keras are and how they are used for NNs*

Next lecture:

- *Loss functions, back-propagation and gradient descent*
- *Train-validation-test split in neural networks*
- *Our first neural network coding practice*